

[Github Repository](#)

مهدی جمال خواه

Last Commit SHA: 0bc1aedcfb0d2e8ac94bdf72efb1d363c8c73466

810100111

سوال اول

کاربرد متدهای Assume این است که اگر شرایط مورد نیاز آن تست برقرار بود اجرا شود در غیر این صورت Ignore خواهد شد. اگر ما از متدهای Assume استفاده نکنیم ممکن است که تست ما Fail شود در حالی که دلیل آن چیزی غیر از آن بوده که تست می خواسته بیازماید. به عنوان مثال فرض کنید ما می خواهیم بخشی از منطق برنامه را تست کنیم که برای اجرای آن نیاز داریم به پایگاه داده وصل باشیم، به همین خاطر در ابتدای تست با استفاده از متد assertTrue فرض می کنیم که پایگاه داده در دسترس هست.

```
@Test
void testMethod() {
    assertTrue(isDatabaseAvailable());
    // Run test
}
```

اگر آرگومان assertTrue درست (True) باشد بعد از این جمله نیز اجرا می شود ولی اگر False باشد تست اجرا نخواهد شد و Ignore می شود. یکی دیگر از کاربردهای این متد زمانی است که ورودی تست را به صورت رندوم تولید می کنیم اما می خواهیم که ورودی یک سری شرایط را دارا باشد (مثلا منفی نباشد).

سوال دوم

در Unit Test ما اطمینان نمی دهیم که کد درست است، بلکه می گوییم که این مورد خاص را به درستی انجام می دهد. حال فرض کنید که برای Thread Safe بودن یک کد چند مدل از ترتیب اجرا شدن Thread ها را بررسی کنیم و درست باشد، اما نمی توانیم اطمینان بدهیم که همیشه درست خواهد بود. چون ممکن است بار بعد Thread ها با ترتیب دیگری اجرا شوند و کد درست کار نکند. برای مطمئن شدن از Thread Safe بودن یک کد باید تمام Interleaving ها بررسی شوند یا این که با روش های Formal درستی آن را اثبات کرد.

سوال سوم

ما دوست داریم که تا جایی که امکان پذیر است تست‌ها کاملاً خودکار باشند، تا اجرای آن سریع‌تر باشد؛ که منجر به هزینه کمتر برای اجرای تست می‌شود. از طرفی اگر تست‌ها زمان‌بر باشند نمی‌توانیم بعد از هر تغییری آن را اجرا کنیم و تعداد بار تستی که انجام می‌شود بسته به اینکه چقدر زمان‌بر باشد کاهش می‌یابد. و این باعث می‌شود که کیفیت کد پایین بیاید.

حال اگر به جای استفاده از Assert از Print استفاده کنیم دیگر تست‌ها کاملاً خودکار نخواهند بود و Assertion آن‌ها باید توسط انسان انجام شود و تمام مشکلات گفته شده در پاراگراف قبل را به همراه خواهد داشت.

سوال چهارم

(الف)

از آنجا که اسم ورودی متد Process را BadInput گذاشته است یعنی این تست انتظار دارد که متد process یک Exception پرتاب کند، اول اینکه اگر این تابع Exception پرتاپ کند تست Fail می‌کند نه اینکه Pass شود. مورد دوم اینکه ما در تست انتظار داریم که مشخص باشد چه چیزی را می‌خواهد Verify کند و به صورت نرمال تست تمام شود و بگوید که آیا این ویژگی مشخص درست کار می‌کند یا خیر، اما این تست با اصل گفته شده ناسازگار است و نه تنها اشتباه هست بلکه خوانایی کافی را نیز ندارد.

دو مشکل دیگر این تست که شاید هدف سوال نبوده باشد این است که، اول کلید واژه expects اشتباه است و باید به جای آن از throws استفاده کند. مورد دوم، نام‌گذاری تست است که هیچ اطلاعاتی به ما نمی‌دهد و باید دقیق مشخص کند که چه چیزی را در چه شرایطی تست می‌کند و انتظار چه خروجی‌ای را دارد.

یک راه‌حل برای آن به شکل زیر می‌تواند باشد:

@Test

```
public void Process_BadInput_ExceptionThrown () {  
    Int badInput = 0;  
    assertThrows(Exception.class, () -> {  
        new AnotherClass().process(badInput);  
    });  
}
```

(ب)

در این تست ترتیب اجرا شدن تست‌ها مهم هستند، یعنی اجرا شدن تست‌ها از هم مستقل نیستند و این مشکل ایجاد خواهد کرد. مثلاً فرض کنید بخواهیم تست دوم را به تنهایی اجرا کنیم، در این صورت تست Fail می‌شود در حالی که کد SUT مشکلی ندارد و مشکل از تست اشتباه هست.

همچنین اینکار نگهداری تست‌ها را هم سخت می‌کند فرض کنید یک تست دیگر را به هر دلیلی قبل از این تست بنویسم یا اینکه توی اجرا شدن تست‌ها، سیستم به ترتیب تست‌ها را اجرا نکند و در نتیجه قبل از اجرا

شدن تابع `testSubsequentAccumulate` آن Fixture ای که تست انتظار دارد ممکن است تغییر کرده باشد و تست به اشتباه Fail شود.
یک راه حل این است که تست دوم نیز Fixture را ست بکند:

```
public class TestCalculator() {
    Calculator fixture = Calculator.getInstance();

    @Test
    public void testAccumulate() {
        fixture.setInitialValue(0);
        int result = fixture.accumulate(50);
        Assertions.assertEquals(50, result);
    }

    @Test
    public void testSebsequentAccumulate() {
        fixture.setInitialValue(0);
        fixture.accumulate(50);
        int result = fixture.accumulate(100);
        Assertions.assertEquals(150, result);
    }
}
```

یک راه حل بهتر این است که یک Setup برای کلاس مورد نظر تعریف کنیم:

```
public class TestCalculator() {
    Calculator fixture;

    @BeforeEach
    public void setUp() {
        fixture = Calculator.getInstance();
        fixture.setInitialValue(0);
    }

    @Test
    public void testAccumulate() {
        int result = fixture.accumulate(50);
        Assertions.assertEquals(50, result);
    }

    @Test
    public void testSebsequentAccumulate() {
        fixture.accumulate(50);
        int result = fixture.accumulate(100);
        Assertions.assertEquals(150, result);
    }
}
```

} }